Elementary Operations on Quasi-Toeplitz Matrices

PCCA6

Ali Arda BARUT Rafael LLAMAS

May 27, 2026

Context

Toeplitz Matrix A_1

$$A_1 = \begin{pmatrix} 4 & 3 & 2 & 1 \\ 5 & 4 & 3 & 2 \\ 6 & 5 & 4 & 3 \\ 7 & 6 & 5 & 4 \end{pmatrix}$$

Where do we see these structured matrices?

- Sylvester matrix
- Diagonal gradients
- Quantum channel calculations in classical quantum simulations
- Digital Signal Processing
- ...

Toeplitz Matrices

Definition

A matrix $A \in \mathbb{K}^{n \times m}$ is **Toeplitz** if $A_{i,j} = A_{i+1,j+1}$ for all valid i, j.

$$A = \begin{pmatrix} a_0 & a_{-1} & \cdots & & & & & a_{-(m-1)} \\ a_1 & a_0 & a_{-1} & & & & & a_{1-(m-1)} \\ \vdots & a_1 & a_0 & \ddots & & & & a_{2-(m-1)} \\ \vdots & & \ddots & \ddots & \ddots & & & \vdots \\ a_{n-1} & \cdots & \cdots & a_1 & a_0 & a_{-1} & \cdots & a_{(n-1)-(m-1)} \end{pmatrix}$$

$$A_{i,j} = a_{i-j}, \quad 0 \leq i < n, \ 0 \leq j < m$$

Examples of Toeplitz Matrices

$$A_1 = \begin{pmatrix} 4 & 3 & 2 & 1 \\ 5 & 4 & 3 & 2 \\ 6 & 5 & 4 & 3 \\ 7 & 6 & 5 & 4 \end{pmatrix} \qquad A_2 = \begin{pmatrix} 120 & 432 & 584 & 394 & 159 \\ 312 & 120 & 432 & 584 & 394 \\ 985 & 312 & 120 & 432 & 584 \end{pmatrix}$$

Displacement

Displacement Operator

$$\nabla A = A - ZAZ^T = GH^T$$

For Toeplitz: $\mathrm{rank}(\nabla A) \leq 2$

Shift Matrix

$$Z = \begin{pmatrix} 0 & & & \\ 1 & & & \\ & \ddots & & \\ & & 1 & 0 \end{pmatrix} \quad Z^T = \begin{pmatrix} 0 & 1 & & \\ & & \ddots & \\ & & & 1 \\ & & & 0 \end{pmatrix}$$

Displacement and Generators

Displacement Operator & Rank

- **Operator:** $\nabla A = A - ZAZ^T$ (Displacement Rank: $\alpha = \text{rank}(\nabla A)$).
- **Generators:** We can represent A by factoring $\nabla A = GH^\top$, where G and H are $n \times \alpha$ matrices.
- **Toeplitz property:** For any Toeplitz matrix, $\alpha \leq 2$.

Example: Displacement of a Toeplitz Matrix

$$A_1 = \begin{pmatrix} 4 & 3 & 2 & 1 \\ 5 & 4 & 3 & 2 \\ 6 & 5 & 4 & 3 \\ 7 & 6 & 5 & 4 \end{pmatrix} \longrightarrow \nabla A_1 = \begin{pmatrix} 4 & 3 & 2 & 1 \\ 5 & 0 & 0 & 0 \\ 6 & 0 & 0 & 0 \\ 7 & 0 & 0 & 0 \end{pmatrix}$$

Only the first row and column remain non-zero $\implies \text{rank}(\nabla A_1) = 2$.

Example - Displacement

Displacement Matrix A_1

$$\nabla A_1 = \begin{pmatrix} 4 & 3 & 2 & 1 \\ 5 & 0 & 0 & 0 \\ 6 & 0 & 0 & 0 \\ 7 & 0 & 0 & 0 \end{pmatrix}$$

Displacement Matrix A_2

$$\nabla A_2 = \begin{pmatrix} 120 & 432 & 584 & 394 & 159 \\ 312 & 0 & 0 & 0 & 0 \\ 985 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Generators G_1 and H_1

$$G_1 = \begin{pmatrix} 4 & 0 \\ 5 & 1 \\ 6 & 0 \\ 7 & 0 \end{pmatrix}$$

$$H_1^T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 3 & 2 & 1 \end{pmatrix}$$

Generators G_2 and H_2

$$G_2 = \begin{pmatrix} 120 & 0 \\ 312 & 1 \\ 985 & 0 \end{pmatrix}$$

$$H_2^T = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 432 & 584 & 394 & 159 \end{pmatrix}$$

Reconstruction

Reconstruction

$$A = \sum_{k=1}^{\alpha} L(g_k)\, U(h_k)$$

Defining L and U are Lower and Upper matrices respectively:

$$L(g_k) = \begin{bmatrix} g_{k,1} & 0 & 0 & 0 \\ g_{k,2} & g_{k,1} & 0 & 0 \\ g_{k,3} & g_{k,2} & g_{k,1} & 0 \\ g_{k,4} & g_{k,3} & g_{k,2} & g_{k,1} \end{bmatrix} \qquad U(h_k) = \begin{bmatrix} h_{k,1} & h_{k,2} & h_{k,3} & h_{k,4} \\ 0 & h_{k,1} & h_{k,2} & h_{k,3} \\ 0 & 0 & h_{k,1} & h_{k,2} \\ 0 & 0 & 0 & h_{k,1} \end{bmatrix}$$

Example - Reconstruction

$$A = L(g^{(1)})U(h^{(1)}) + L(g^{(2)})U(h^{(2)})$$

$$L(g^{(1)})U(h^{(1)}) = \begin{pmatrix} 4 & 0 & 0 & 0 \\ 5 & 4 & 0 & 0 \\ 6 & 5 & 4 & 0 \\ 7 & 6 & 5 & 4 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 4 & 0 & 0 & 0 \\ 5 & 4 & 0 & 0 \\ 6 & 5 & 4 & 0 \\ 7 & 6 & 5 & 4 \end{pmatrix}$$

$$L(g^{(2)})U(h^{(2)}) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 3 & 2 & 1 \\ 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 3 & 2 & 1 \\ 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Complexity - Reconstruction

Reconstruction

$$A = \sum_{k=1}^{\alpha} L(g_k)\, U(h_k)$$

Complexities

Naive: $O(n^3)$
Polynomial Product : $O(n^2)$

Multiplication of Toeplitz-like Matrices

Generator Multiplication with Vectors

$$A = \sum_{k=1}^{\alpha} L(g_k)\, U(h_k) V$$

Complexities

Naive: $O(n^3)$
Polynomial Products : $O(\alpha M(n))$

Multiplication of Toeplitz-like Matrices

Generator Multiplication

$$
\begin{aligned}
A &\simeq (T, U), & B &\simeq (G, H) \\
W &= ZAZ^\top G, & V &= B^\top U \\
\mathrm{LastCol} &= (0, \ldots, 0, 1)^\top, & a &= ZA\,\mathrm{LastCol} \\
b &= ZB^\top \mathrm{LastCol}, & C &\simeq (T \mid W \mid a,\ V \mid H \mid b)
\end{aligned}
$$

Complexities

Naive: $\mathcal{O}(n^3)$
Concatenation: $\mathcal{O}(\alpha^2 M(n))$

Addition of Toeplitz-like Matrices

Generator Addition

$$A \simeq (T, U), B \simeq (G, H) \Rightarrow A + B \simeq (T|G, U|H)$$

Complexities

Naive: $O(n^2)$
Concatenation : $O(\alpha n)$

Example - Addition

$$C = 2A \qquad with \qquad A_1 = \begin{pmatrix} 4 & 3 & 2 & 1 \\ 5 & 4 & 3 & 2 \\ 6 & 5 & 4 & 3 \\ 7 & 6 & 5 & 4 \end{pmatrix}, G = \begin{pmatrix} 4 & 0 \\ 5 & 1 \\ 6 & 0 \\ 7 & 0 \end{pmatrix}, H^T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 3 & 2 & 1 \end{pmatrix}$$

$$T = \begin{pmatrix} 4 & 0 & 4 & 0 \\ 5 & 1 & 5 & 1 \\ 6 & 0 & 6 & 0 \\ 7 & 0 & 7 & 0 \end{pmatrix}, U = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 3 & 0 & 3 & 0 \\ 2 & 0 & 2 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

Example - Addition

$$C = L(g^{(1)})U(h^{(1)}) + L(g^{(2)})U(h^{(2)}) + L(g^{(3)})U(h^{(3)}) + L(g^{(4)})U(h^{(4)})$$

$$\begin{pmatrix} 4 & 0 & 0 & 0 \\ 5 & 4 & 0 & 0 \\ 6 & 5 & 4 & 0 \\ 7 & 6 & 5 & 4 \end{pmatrix} + \begin{pmatrix} 0 & 3 & 2 & 1 \\ 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \end{pmatrix} + \begin{pmatrix} 4 & 0 & 0 & 0 \\ 5 & 4 & 0 & 0 \\ 6 & 5 & 4 & 0 \\ 7 & 6 & 5 & 4 \end{pmatrix} + \begin{pmatrix} 0 & 3 & 2 & 1 \\ 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$= \begin{pmatrix} 8 & 6 & 4 & 2 \\ 10 & 8 & 6 & 4 \\ 12 & 10 & 8 & 6 \\ 14 & 12 & 10 & 8 \end{pmatrix}$$

Compression

Operations like addition and multiplication artificially increase the rank of the generator matrices.

$$T = \begin{pmatrix} 4 & 0 & 4 & 0 \\ 5 & 1 & 5 & 1 \\ 6 & 0 & 6 & 0 \\ 7 & 0 & 7 & 0 \end{pmatrix} U = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 3 & 0 & 3 & 0 \\ 2 & 0 & 2 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

represents:

$$\begin{pmatrix} 8 & 6 & 4 & 2 \\ 10 & 8 & 6 & 4 \\ 12 & 10 & 8 & 6 \\ 14 & 12 & 10 & 8 \end{pmatrix}$$

Compression

Definition

Matrices $G \in \mathbb{R}^{n\times\alpha}$ and $H \in \mathbb{R}^{n\times\alpha}$ represent a matrix $\nabla A = GH^T$.

Goal: Find two matrices $G' \in \mathbb{R}^{n\times r}$ and $H' \in \mathbb{R}^{n\times r}$ for minimal r such that:

$$G'H'^T = GH^T$$

Compression

Algorithm 1 Generator Compression

```
function GR_MAT_GENERATOR_COMPRESS_AUX(G, H)
    P, L, U ← LU_Decomposition(G^T)                    ▷ O(α^2 n)
    r ← Number of non-zero rows in U                   ▷ O(α)
    if r < 1 then
        return Generators of a 0 matrix                ▷ O(1)
    end if
    G_D ← (U_[0:r−1],*)^T                              ▷ O(αn)
    H_D ← HP^T × L_*,[0:r−1]                           ▷ O(α^2 n)
    return G_D, H_D                                    ▷ O(1)
end function
```

Giving us a total complexity of $\mathcal{O}(\alpha^2 n)$

Inversion

Inverting a Toeplitz matrix represented by generators allows us to efficiently solve systems. But can also be used in:

- Image unblurring and de-echo.
- Speech compression and financial forecasting.

Inversion

Inverting a Toeplitz matrix represented by generators allows us to efficiently solve systems.
We can apply a divide-and-conquer algorithm, aiming to reduce the complexity from $O(n^3)$ to $O(\alpha^2 M(n) \log n)$.

Inversion

Algorithm 2 Strassen-type Inversion for Generators: $\textsc{StrassenInv}(G_A, H_A)$

if $n = 1$ **then return** scalar inverse (G_D, H_D)
end if ▷ **Step 1: Base Case**
$G_a, H_a, \ldots, G_d, H_d \leftarrow \textsc{Split}(G_A, H_A)$ ▷ **Step 2: Quadrants**
$G_e, H_e \leftarrow \textsc{StrassenInv}(G_a, H_a)$ ▷ **Step 3: Recursion 1 ($e := a^{-1}$)**
$G_{ce}, H_{ce} \leftarrow \mathcal{C}(G_c, H_c \otimes G_e, H_e)$ ▷ **Step 4: Schur Complement**
$G_{eb}, H_{eb} \leftarrow \mathcal{C}(G_e, H_e \otimes G_b, H_b)$
$G_{ceb}, H_{ceb} \leftarrow \mathcal{C}(G_c, H_c \otimes G_{eb}, H_{eb} \times -1)$
$G_S, H_S \leftarrow \mathcal{C}(G_d, H_d \oplus G_{ceb}, H_{ceb})$ ▷ $S := d - ceb$
$G_t, H_t \leftarrow \textsc{StrassenInv}(G_S, H_S)$ ▷ **Step 5: Recursion 2 ($t := S^{-1}$)**
$G_{ebt}, H_{ebt} \leftarrow \mathcal{C}(G_{eb}, H_{eb} \otimes G_t, H_t)$ ▷ **Step 6: Strassen Assembly**
$G_{tce}, H_{tce} \leftarrow \mathcal{C}(G_t, H_t \otimes G_{ce}, H_{ce})$
$G_{ebtce}, H_{ebtce} \leftarrow \mathcal{C}(G_{ebt}, H_{ebt} \otimes G_{ce}, H_{ce})$
$G_x, H_x \leftarrow \mathcal{C}(G_e, H_e \oplus G_{ebtce}, H_{ebtce})$ ▷ $x := e + ebtce$
$G_y, H_y \leftarrow \textsc{Negate}(G_{ebt}, H_{ebt})$; $\quad G_z, H_z \leftarrow \textsc{Negate}(G_{tce}, H_{tce})$
$G_{Inv}, H_{Inv} \leftarrow \textsc{Pack}(G_x, H_x, G_y, H_y, G_z, H_z, G_t, H_t)$ ▷ **Step 7: Pack**
return $\mathcal{C}(G_{Inv}, H_{Inv})$

Notation: $\mathcal{C}(\cdot)$ = *Compress*, $\otimes$ = *Multiply Generators*, $\oplus$ = *Add Generators*

Inversion

Algorithm 3 Strassen-type Inversion for Generators: $\textsc{StrassenInv}(G_A, H_A)$

if $n = 1$ **then return** scalar inverse (G_D, H_D)
end if ▷ **Step 1: Base Case**
$G_a, H_a, \dots, G_d, H_d \leftarrow \textsc{Split}(G_A, H_A)$ ▷ **Step 2: Quadrants**
$G_e, H_e \leftarrow \textsc{StrassenInv}(G_a, H_a)$ ▷ **Step 3: Recursion 1 ($e := a^{-1}$)**
$G_{ce}, H_{ce} \leftarrow \mathcal{C}(G_c, H_c \otimes G_e, H_e)$ ▷ **Step 4: Schur Complement**
$G_{eb}, H_{eb} \leftarrow \mathcal{C}(G_e, H_e \otimes G_b, H_b)$
$G_{ceb}, H_{ceb} \leftarrow \mathcal{C}(G_c, H_c \otimes G_{eb}, H_{eb} \times -1)$
$G_S, H_S \leftarrow \mathcal{C}(G_d, H_d \oplus G_{ceb}, H_{ceb})$ ▷ $S := d - ceb$
$G_t, H_t \leftarrow \textsc{StrassenInv}(G_S, H_S)$ ▷ **Step 5: Recursion 2 ($t := S^{-1}$)**
$G_{ebt}, H_{ebt} \leftarrow \mathcal{C}(G_{eb}, H_{eb} \otimes G_t, H_t)$ ▷ **Step 6: Strassen Assembly**
$G_{tce}, H_{tce} \leftarrow \mathcal{C}(G_t, H_t \otimes G_{ce}, H_{ce})$
$G_{ebtce}, H_{ebtce} \leftarrow \mathcal{C}(G_{ebt}, H_{ebt} \otimes G_{ce}, H_{ce})$
$G_x, H_x \leftarrow \mathcal{C}(G_e, H_e \oplus G_{ebtce}, H_{ebtce})$ ▷ $x := e + ebtce$
$G_y, H_y \leftarrow \textsc{Negate}(G_{ebt}, H_{ebt})$; $G_z, H_z \leftarrow \textsc{Negate}(G_{tce}, H_{tce})$
$G_{Inv}, H_{Inv} \leftarrow$ **Pack**$(G_x, H_x, G_y, H_y, G_z, H_z, G_t, H_t)$ ▷ Step 7: Pack
return $\mathcal{C}(G_{Inv}, H_{Inv})$

Notation: $\mathcal{C}(\cdot)$ = *Compress*, $\otimes$ = *Multiply Generators*, $\oplus$ = *Add Generators*

Inversion: Boundary Bleed

$$A = \left(\begin{array}{ccc|ccc} a_{11} & a_{12} & a_{13} & b_{11} & b_{12} & b_{13} \\ a_{21} & a_{22} & a_{23} & b_{21} & b_{22} & b_{23} \\ a_{31} & a_{32} & a_{33} & b_{31} & b_{32} & b_{33} \\ \hline c_{11} & c_{12} & c_{13} & d_{11} & d_{12} & d_{13} \\ c_{21} & c_{22} & c_{23} & d_{21} & d_{22} & d_{23} \\ c_{31} & c_{32} & c_{33} & d_{31} & d_{32} & d_{33} \end{array}\right)$$

Inversion: Split & Pack

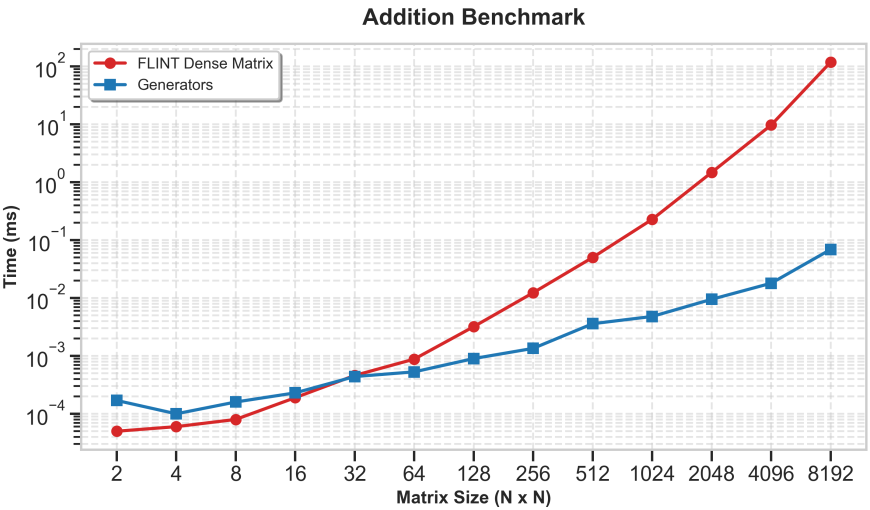
Quadrants of $\nabla A = A - ZAZ^T$

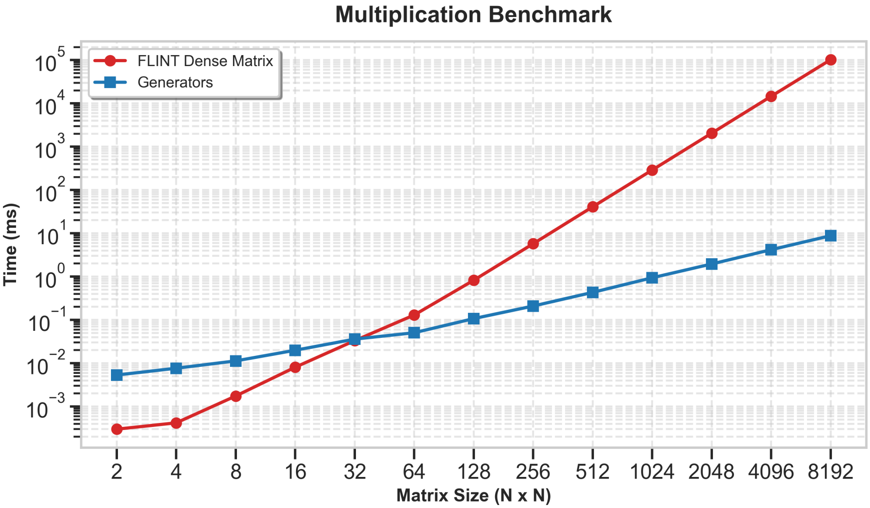
Let A be partitioned into four $n \times n$ quadrants, and Z_{2n} be the shift matrix:

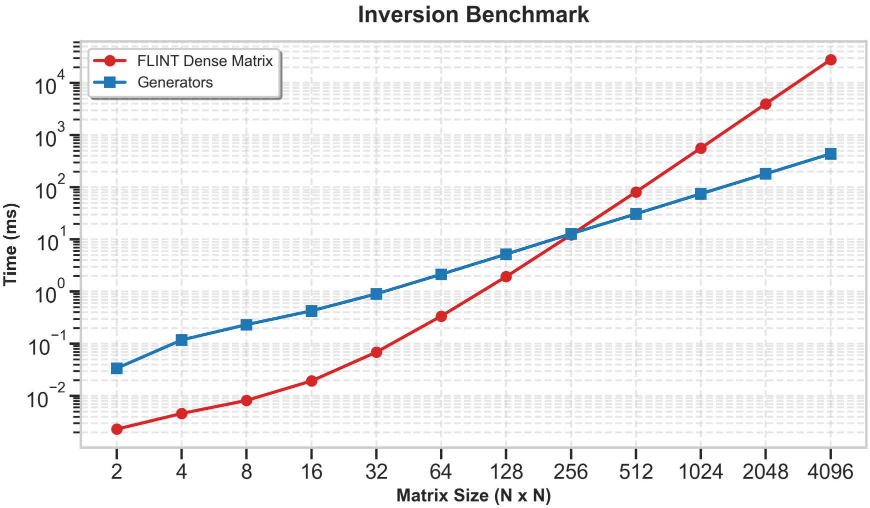
$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \qquad Z_{2n} = \begin{pmatrix} Z_n & 0 \\ e_1 e_n^T & Z_n \end{pmatrix}$$

Let E $= e_1 e_n^T$

$$\nabla A = \begin{pmatrix} \nabla a & \nabla b - Z_n a E^T \\ \nabla c - E a Z_n^T & \nabla d - E a E^T - Z_n c E^T - E b Z_n^T \end{pmatrix}$$


Addition Benchmark
FLINT Dense Matrix
Generators
Time (ms)
10^2
10^1
10^0
10^{-1}
10^{-2}
10^{-3}
10^{-4}
2
4
8
16
32
64
128
256
512
1024
2048
4096
8192
Matrix Size (N x N)


Multiplication Benchmark
FLINT Dense Matrix
Generators
Time (ms)
10^5
10^4
10^3
10^2
10^1
10^0
10^{-1}
10^{-2}
10^{-3}
2
4
8
16
32
64
128
256
512
1024
2048
4096
8192
Matrix Size (N x N)


Inversion Benchmark
FLINT Dense Matrix
Generators
Time (ms)
10^{4}
10^{3}
10^{2}
10^{1}
10^{0}
10^{-1}
10^{-2}
2
4
8
16
32
64
128
256
512
1024
2048
4096
Matrix Size (N x N)

Thank you